

The 4-Layer Eval Stack: Coverage vs Cost, by Hand

Why no single eval layer is both cheap and complete — every percentage point of coverage and cent of cost counted, for one defect.

What this document does. It takes the one stack from Module 3.8 — the *4-Layer Eval Stack* of **code scorers** → **LLM judges** → **humans** → **benchmarks** — fixes a detection rate and a cost fraction for each layer, and then computes, layer by layer, how much of a defect the stack catches and what that catching costs. Nothing is asserted. The module quotes a healthy program as ~90% code, ~9% judges, ~1% humans, plus quarterly benchmarks; we rebuild *why* those ratios fall out of the coverage-per-cost arithmetic. We also derive the cascading-confidence rule the module leans on when it prices a multi-agent chain (router × specialist). Every number below is reproduced by the small Python program in Appendix A.

Where this sits. This is **Topic 1 of Module 3.8** — the capstone decision guide for Track 3 (Agent Evaluation). Its companions: Module 3.2 (multi-agent evaluation — where the cascading-confidence product first bites) and Module 3.4 (cost & latency — the same coverage-vs-cost trade, seen from the spend side).

Concepts covered, in order: cascading confidence as a product of stage accuracies · a defect’s combined coverage $1 - \prod(1 - d_i)$ for independent layers · total cost as $\sum c_i$ · marginal coverage per added layer · diminishing returns · the cost-weighted “add a layer while $\Delta\text{cov}/\Delta\text{cost} > \tau$ ” rule · why code is 90% of the value at 1% of the cost and humans are the opposite · what benchmarks buy that coverage math does not price.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one defect, four layers that might catch it

An eval program is a stack of filters. A run produces some *defect* — a wrong answer, a bad tool call, a cost spike, a leaked secret — and each layer of the stack has some chance of *catching* it before it ships. The module names four layers, cheapest first:

- **Code scorers** — deterministic checks (task completion, tool F1, step budgets, cost percentiles, PII regex). Run every commit, cost $\approx$ \$0.
- **LLM judges** — trajectory / tool-call / pairwise judges for what code cannot quantify. Run nightly, $\sim$ \$0.05 a run.
- **Humans** — the gold set and judge calibration. The ground truth, at \$1–\$10 an item, run weekly.
- **Benchmarks** — public suites + a domain holdout. Insurance against quiet drift, run quarterly.

To count by hand we assign each layer two numbers: a *detection rate* d_i (the probability that layer, alone, catches a given defect) and a *cost fraction* c_i (its share of the eval budget). These are illustrative round numbers, chosen so the products are followable on paper; the mechanics are identical at any values.

Layer	Symbol	Detection d_i	Cost fraction c_i
Code scorers	layer 1	0.90	0.01
LLM judges	layer 2	0.60	0.05
Humans	layer 3	0.50	0.20
Benchmarks	layer 4	— (insurance)	variable

Notation. d_i is a per-layer catch probability; a layer *misses* with probability $1 - d_i$. Layers are treated as *independent* detectors of the same defect (a strong assumption we revisit in the honest-word section). Coverage is the fraction of defects the stack catches; cost is the cumulative budget fraction spent. We carry four decimals throughout.

Intuition

A stack of eval layers is a stack of nets under a tightrope walker. No single net catches every fall — but a faller only reaches the floor if *every* net under them happens to have a hole in the right spot. Add nets and the surviving-fall probability multiplies down fast. The catch is that each net costs more to hang than the one above it, while plugging an ever-smaller share of the remaining holes. The whole document is that one trade-off, made numeric.

2 Step 1 — cascading confidence: accuracies multiply

Before the stack, the same multiply-the-survivors logic explains why a multi-agent *system* is less reliable than any of its parts. If a request must pass through several stages in series — router, then specialist, then synthesiser — and each stage succeeds independently with probability a_i , the whole system succeeds only if *every* stage does:

$$S = \prod_i a_i. \quad (1)$$

This is the module’s headline multi-agent figure. A router that picks the right specialist 94% of the time, feeding a specialist that is right 90% of the time, yields

$$S = 0.94 \times 0.90 = 0.846.$$

Not 0.94, not 0.90 — 0.846. The product is always smaller than its smallest factor. Tabulating a few chains makes the erosion vivid:

Chain	Stages	Per-stage	System S
Router → specialist	2	0.94, 0.90	0.8460
Router → specialist → synth	3	0.94, 0.90, 0.95	0.8037
Three good stages	3	0.95 each	0.8574
Five good stages	5	0.95 each	0.7738
Five <i>excellent</i> stages	5	0.99 each	0.9510

Sample check, row 1: $0.94 \times 0.90 = 0.846$. ✓ Row 4: $0.95^5 = 0.7738$. ✓ Notice the last two rows: five stages at 95% each lands at only 77%, but lifting each stage to 99% recovers 95%. The lesson

the module draws is that in a chain, *stage* accuracy must be far higher than the *system* target you want — and that is exactly why you evaluate each stage in isolation, not just the end-to-end answer.

Mini-example — this operation in isolation

The product compounds the wrong way for you. Two 95% stages give $0.95^2 = 0.9025$ — you have already lost 5 points off the weaker bound. Each additional 95% stage multiplies what survives by 0.95 again, so reliability decays *geometrically* in the number of stages. Halving the per-stage error (from 5% to 2.5%) roughly halves the system error too — which is why squeezing one bad stage pays more than adding redundancy around a good one.

3 Step 2 — combined coverage: misses multiply

The same product, run on the *miss* probabilities, gives the stack’s coverage. The stack fails to catch a defect only if *every* layer independently misses it. With layer misses $1 - d_i$,

$$\text{coverage} = 1 - \prod_i (1 - d_i) \quad (2)$$

Working the three catching layers in order, $d = 0.90, 0.60, 0.50$:

After adding	d_i	$1 - d_i$	cumulative miss	coverage	caught
Code scorers	0.90	0.10	0.1000	0.9000	90.00%
+ LLM judges	0.60	0.40	0.0400	0.9600	96.00%
+ Humans	0.50	0.50	0.0200	0.9800	98.00%

The cumulative-miss column is just the running product: 0.10, then $0.10 \times 0.40 = 0.04$, then $0.10 \times 0.40 \times 0.50 = 0.02$. So the full three-layer stack *misses* 2% of defects and **catches** 98%. Sample check: $0.10 \times 0.40 \times 0.50 = 0.02 \Rightarrow 1 - 0.02 = 0.98$. ✓

Intuition

Coverage is built by *killing the survivors*, not by adding up catches. Code already takes the miss rate down to 10%. Judges only get to work on *that surviving* 10% — catching 60% of it removes 6 points, not 60. Humans work on the 4% that is left, and remove 2 more. Each layer is just as capable as the headline d_i says; it simply has less and less left to find. That shrinking remainder is the entire reason the layers get cheaper-per-point as you climb — it is structural, not a quirk of these numbers.

4 Step 3 — the cost side, and marginal returns

Coverage is only half the ledger. Each layer also adds cost. Total cost is the simple sum $\text{Cost} = \sum_i c_i$, but the figure that drives the *decision* is the **marginal** one: how much new coverage the next layer buys, divided by what it costs.

$$\text{value of layer } i = \frac{\Delta \text{coverage}_i}{\Delta \text{cost}_i} = \frac{\text{coverage}_i - \text{coverage}_{i-1}}{c_i}.$$

Stacking the layers and carrying both columns:

Layer	coverage	Δcov	c_i	cum. cost	$\frac{\Delta\text{cov}}{\Delta\text{cost}}$
Code scorers	0.9000	0.9000	0.01	0.01	90.0
+ LLM judges	0.9600	0.0600	0.05	0.06	1.20
+ Humans	0.9800	0.0200	0.20	0.26	0.10

Read the last column top to bottom: $90.0 \rightarrow 1.2 \rightarrow 0.1$. The marginal value of a layer falls by roughly two orders of magnitude across three layers. Sample check, judges: $\Delta\text{cov} = 0.96 - 0.90 = 0.06$, $\Delta\text{cost} = 0.05$, ratio $= 0.06/0.05 = 1.2$. ✓ Humans: $0.02/0.20 = 0.10$. ✓

Mini-example — this operation in isolation

Code’s marginal value is 90 — it returns 90 points of coverage per unit of cost spent. That is the quantitative form of “90% of the value at 1% of the cost.” Compare it to humans at 0.10: humans return one point of coverage for every *ten* units of cost. The same layer that is the ground truth (most trustworthy) is also the worst marginal buy for routine defect-catching. Both facts are true at once, and they are not in tension — you are buying different things, which is the next step.

5 Step 4 — the cost-weighted decision rule

The marginal column gives a clean stopping rule. **Add the next layer while its $\Delta\text{cov}/\Delta\text{cost}$ exceeds a threshold τ ; stop when it drops below.** Pick $\tau = 1.0$ (one point of coverage is worth one unit of cost) and walk the table:

Layer	$\Delta\text{cov}/\Delta\text{cost}$	Decision at $\tau = 1.0$
Code scorers	90.0	ADD (overwhelmingly)
LLM judges	1.2	ADD (clears the bar)
Humans	0.10	skip for coverage

So a purely coverage-driven budget buys code and judges and *stops*. That is not a mistake in the math — it is the math telling you the truth: humans are not bought for marginal defect coverage. They are bought for something the coverage model does not price.

Watch out

The rule says “skip humans,” yet every healthy program runs them. The resolution is that d_i for humans understates their job. Humans are the *ground truth* that calibrates the judges’ κ and anchors the gold set — without them, the judges’ $d = 0.60$ is unverified and could silently drift to 0.30. Likewise **benchmarks** have no d_i at all in this model: they catch *slow distributional drift* that no single-run filter can see, which is why the module files them under “insurance,” run quarterly. The lesson: a coverage/cost optimiser will always try to delete your most expensive layer. Keep it anyway — for the variance it controls, not the mean it improves. The healthy split ($\sim 90/9/1$ + quarterly) is exactly this rule, with humans funded at a floor for calibration rather than at their marginal coverage.

6 Step 5 — value density: 90% of value at 1% of cost

To make the headline intuition exact, score each layer in *isolation* on detection-per-cost d_i/c_i :

Layer	detection d_i	cost c_i	d_i/c_i
Code scorers	0.90	0.01	90.0
LLM judges	0.60	0.05	12.0
Humans	0.50	0.20	2.5

Code’s value density is 90; humans’ is 2.5 — a $36\times$ gap. Check: $0.90/0.01 = 90$, $0.50/0.20 = 2.5$, ratio $90/2.5 = 36$. ✓ This is why the ratios in the module are not arbitrary taste: you spend the most budget where each dollar buys the most detection (code), a little where it buys moderate detection on what code missed (judges), and a trickle on the irreplaceable-but-costly ground truth (humans). The pyramid is upside-down in cost *because* it is right-side-up in value density.

Intuition

Code metrics and humans are mirror images. Code is cheap, broad, and shallow — it catches the 90% of defects that have a deterministic signature (wrong tool name, over-budget step count, a regex-matchable secret) for almost nothing, but it is blind to anything requiring judgment. Humans are expensive, narrow, and deep — they catch the subtle 1% that needs taste, but you could never afford to run them on every commit. The stack exists precisely because these two properties never live in the same layer: *no single layer is both cheap and complete*. You assemble completeness out of cheap-and-shallow plus expensive-and-deep.

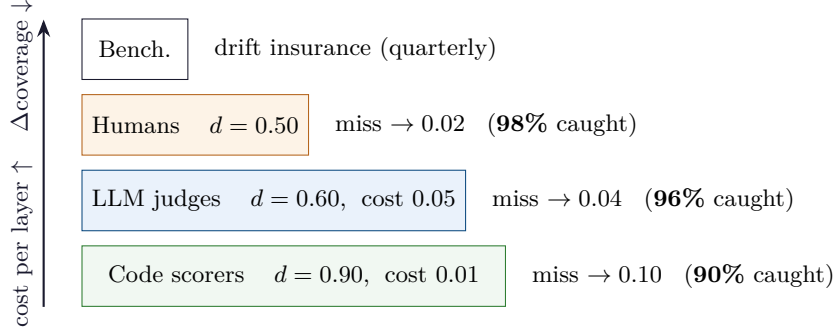
7 What changes these numbers — and what does not

The detection rates are workload-specific. If your defects are mostly subtle reasoning errors rather than malformed tool calls, code’s d drops and judges’ d rises — the marginal table reshuffles, and judges may deserve a bigger budget share. The *structure* does not move: coverage is still $1 - \prod(1 - d_i)$ and marginal value still falls as the remainder shrinks.

Independence is optimistic. We assumed the layers miss independently. In reality a defect that fools the code scorer (say, a plausible-but-wrong answer) often also fools the judge for the same reason. Correlated misses make the true combined coverage *lower* than $1 - \prod(1 - d_i)$ — the product is an upper bound on how much overlapping layers help. This is the quantitative case for *diverse* layers: a human and a cross-family judge miss different things than your code does, so they buy more real coverage than a second code scorer would.

What never changes: marginal coverage per added layer falls monotonically as long as each layer attacks the same shrinking remainder, while cost-per-layer rises. So a cheap, broad bottom layer plus a thin, expensive top layer will always dominate any flat, single-layer program on coverage-per-dollar. That shape — many cheap catches, few expensive ones — is the eval stack.

8 The stack, drawn



Box *width* is cost share; box *height position* is the order of stacking. The widest box (code) sits at the bottom and does the most catching for the least money; each box above is narrower in value-per-dollar and plugs a thinner slice of what remains. The picture is Equation (2) read from bottom to top.

9 Eval-economics cheat-sheet

Cascading system success	$S = \prod_i a_i$
Combined defect coverage	$\text{cov} = 1 - \prod_i (1 - d_i)$
Cumulative miss after k layers	$\prod_{i=1}^k (1 - d_i)$
Total cost	$\text{Cost} = \sum_i c_i$
Marginal value of layer i	$\frac{\text{cov}_i - \text{cov}_{i-1}}{c_i}$
Add-a-layer rule	add while $\frac{\Delta\text{cov}}{\Delta\text{cost}} > \tau$
Value density (isolated)	d_i/c_i
Healthy split (module)	$\sim 90\% \text{ code} \cdot 9\% \text{ judges} \cdot 1\% \text{ human} \cdot \text{qtr. bench}$

10 An honest word about these numbers

The detection rates (0.90/0.60/0.50) and cost fractions (0.01/0.05/0.20) are illustrative round numbers, picked so the products land on clean values like 0.02 and 98% — exactly as the cost-by-hand document used round token sizes no one would ship. Your real d_i depend on what your defects look like, and your real c_i depend on annotator wages and traffic volume; change them and the 98% and the 90/1.2/0.1 marginal ladder all move. Two things are *not* illustrative. First, the algebra: independent layers always combine as $1 - \prod(1 - d_i)$, costs always add, and marginal coverage always shrinks against rising cost — so the optimal program is always cheap-and-broad at the bottom, thin-and-expensive at the top. Second, the warning: the independence assumption flatters the stack, and a pure coverage optimiser will always try to cut your humans and benchmarks. Keep them anyway — they buy calibration and drift-insurance the coverage model cannot see. That shape, not these digits, is what to carry into every eval review.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the cascading chains, the stacked-coverage table, the marginal-value ladder, and the decision rule; change the `layers` list to model your own defect profile and budget.

```

1  # eval_stack.py -- reproduces every number in "The 4-Layer Eval Stack".
2
3  # --- Part 1: cascading confidence: system success = product of stage accuracies ---
4  def chain_success(stages):
5      p = 1.0
6      for s in stages:
7          p *= s
8      return p
9
10 chains = [
11     ("router 0.94 -> specialist 0.90", [0.94, 0.90]),
12     ("router 0.94 -> specialist 0.90 -> synth 0.95", [0.94, 0.90, 0.95]),
13     ("3 stages each 0.95", [0.95, 0.95, 0.95]),
14     ("5 stages each 0.95", [0.95]*5),
15     ("5 stages each 0.99", [0.99]*5),
16 ]
17 print("== cascading confidence ==")
18 for name, st in chains:
19     print(f"{name:40s} -> {chain_success(st):.4f}")
20
21 # --- Parts 2-5: layers as (name, detection d_i, cost fraction c_i) ---
22 layers = [
23     ("Code scorers", 0.90, 0.01),
24     ("LLM judges", 0.60, 0.05),
25     ("Humans", 0.50, 0.20),
26 ]
27
28 print("\n== combined coverage, stacking layers (cov = 1 - prod(1-d)) ==")
29 miss = 1.0; cum_cost = 0.0; prev_cov = 0.0
30 for name, d, c in layers:
31     miss *= (1 - d)
32     cov = 1 - miss
33     cum_cost += c
34     marg = cov - prev_cov
35     print(f"+{name:13s} d={d:.2f} cost+={c:.2f} -> miss={miss:.4f} "
36           f"cov={cov:.4f} dcov={marg:.4f} cumcost={cum_cost:.2f} "
37           f"dcov/dcost={marg/c:.3f}")
38     prev_cov = cov
39
40 m = (1-0.90)*(1-0.60)*(1-0.50)
41 print(f"worked: miss = 0.10*0.40*0.50 = {m:.4f} -> caught = {1-m:.4f}")
42
43 print("\n== value density (detection per unit cost), isolated ==")
44 for name, d, c in layers:
45     print(f"{name:13s}: {d:.2f} / {c:.2f} = {d/c:.2f} per unit cost")
46
47 print("\n== decision rule: add while dcov/dcost > tau=1.0 ==")
48 miss = 1.0; prev = 0.0
49 for name, d, c in layers:
50     miss *= (1-d); cov = 1-miss; r = (cov-prev)/c
51     print(f"{name:13s}: dcov/dcost = {r:.3f} -> {'ADD' if r>1.0 else 'skip'}")
52     if r > 1.0:
53         prev = cov
54
55 # Expected output:
56 # == cascading confidence ==
57 # router 0.94 -> specialist 0.90 -> 0.8460

```

```
58 # router 0.94 -> specialist 0.90 -> synth 0.95 -> 0.8037
59 # 3 stages each 0.95 -> 0.8574
60 # 5 stages each 0.95 -> 0.7738
61 # 5 stages each 0.99 -> 0.9510
62 #
63 # == combined coverage, stacking layers (cov = 1 - prod(1-d)) ==
64 # +Code scorers d=0.90 cost+=0.01 -> miss=0.1000 cov=0.9000 dcov=0.9000 cumcost=0.01 dcov/
    dcost=90.000
65 # +LLM judges d=0.60 cost+=0.05 -> miss=0.0400 cov=0.9600 dcov=0.0600 cumcost=0.06 dcov/dcost
    =1.200
66 # +Humans d=0.50 cost+=0.20 -> miss=0.0200 cov=0.9800 dcov=0.0200 cumcost=0.26 dcov/dcost
    =0.100
67 # worked: miss = 0.10*0.40*0.50 = 0.0200 -> caught = 0.9800
68 #
69 # == value density (detection per unit cost), isolated ==
70 # Code scorers : 0.90 / 0.01 = 90.00 per unit cost
71 # LLM judges : 0.60 / 0.05 = 12.00 per unit cost
72 # Humans : 0.50 / 0.20 = 2.50 per unit cost
73 #
74 # == decision rule: add while dcov/dcost > tau=1.0 ==
75 # Code scorers : dcov/dcost = 90.000 -> ADD
76 # LLM judges : dcov/dcost = 1.200 -> ADD
77 # Humans : dcov/dcost = 0.100 -> skip
```

— end of document —